

SE446: Big Data Engineering

Week 7B: Spark DataFrames and Spark SQL

Prof. Anis Koubaa

SE 446

Alfaisal University

https://github.com/aniskoubaa/big_data_course

Spring 2026

Today's Agenda

- 1 From RDDs to DataFrames
- 2 Creating DataFrames
- 3 DataFrame Transformations
- 4 Spark SQL
- 5 The Catalyst Optimizer
- 6 Writing Results
- 7 RDD vs DataFrame Decision Guide
- 8 Summary

Outline

- 1 From RDDs to DataFrames
- 2 Creating DataFrames
- 3 DataFrame Transformations
- 4 Spark SQL
- 5 The Catalyst Optimizer
- 6 Writing Results
- 7 RDD vs DataFrame Decision Guide
- 8 Summary

Learning Objectives

By the end of this session you will be able to:

- 1 Understand the DataFrame API and how it differs from RDDs
- 2 Create DataFrames from CSV, JSON, and Parquet files on HDFS
- 3 Apply transformations: `select`, `filter`, `groupBy`, `agg`, `join`
- 4 Query DataFrames using SQL via `createOrReplaceTempView`
- 5 Understand the **Catalyst optimizer** and why DataFrames are faster
- 6 Write DataFrame results back to HDFS in different formats

Dataset

Chicago Crimes (HDFS) + **NYC Yellow Taxi** (examples)

The Problem with Raw RDDs

RDD limitations:

- ✗ No schema — each record is an opaque Python object
- ✗ No query optimizer — Spark executes *exactly* what you wrote
- ✗ Verbose code — manual CSV parsing

```
# RDD: 5 lines of ceremony
rdd.filter(lambda x: x != header) \
    .map(lambda x: x.split(",")) \
    .filter(lambda f: len(f) > 5) \
    .map(lambda f: (f[5], 1)) \
    .reduceByKey(lambda a, b: a + b)
```

The Question

Can we have the power of distributed computing **without** manual parsing and without giving up optimization opportunities?

Enter the DataFrame API

A **DataFrame** is a distributed table with named, typed columns — like a SQL table or a pandas DataFrame, but partitioned across a cluster.

```
# Same query --- ONE line, optimized automatically  
df.groupBy("Primary Type").count().orderBy(col("count").desc())
```

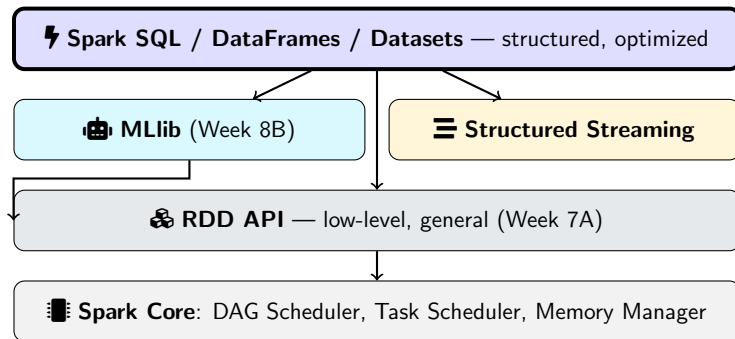
Built on top of RDDs but adds:

- **Schema** (column names + types)
- **Catalyst optimizer** (query planning)
- **Tungsten engine** (binary memory)

Key Insight

DataFrames are **almost always faster** than equivalent RDD code because Catalyst can optimize the execution plan.

The Spark Ecosystem — Layers



Context

MLlib (Week 8) uses DataFrames. Structured Streaming uses DataFrames. **DataFrames are the primary API in modern Spark.**

Outline

- 1 From RDDs to DataFrames
- 2 Creating DataFrames
- 3 DataFrame Transformations
- 4 Spark SQL
- 5 The Catalyst Optimizer
- 6 Writing Results
- 7 RDD vs DataFrame Decision Guide
- 8 Summary

SparkSession — The New Entry Point

SparkSession replaces SparkContext for structured APIs.

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("SE446_CrimeAnalysis") \
    .master("yarn") \
    .getOrCreate()
```

SparkSession vs SparkContext

- SparkSession = structured API entry
- SparkContext still available as `spark.sparkContext`

PySpark Shell

In the PySpark interactive shell, `spark` is **already created** for you — no need to build it manually.

Loading Data into a DataFrame

```
# From CSV on HDFS
df = spark.read.csv(
    "hdfs:///data/chicago_crimes.csv",
    header=True,          # first row = column names
    inferSchema=True      # auto-detect types (costs one extra pass)
)

# From local CSV
df = spark.read.csv("file:///path/to/file.csv", header=True,
                    inferSchema=True)

# From JSON
df = spark.read.json("hdfs:///data/crimes.json")

# From Parquet (columnar --- schema already embedded)
df = spark.read.parquet("hdfs:///data/crimes.parquet")
```

Performance Tip

`inferSchema=True` reads the file **twice** (once to infer types, once to load). For production, define a `StructType` schema explicitly.

Inspecting a DataFrame

```
df.printSchema()      # column names, types, nullable
df.dtypes             # list of (name, type) tuples
df.describe().show()  # count, mean, stddev, min, max
df.show(5)            # print first 5 rows (formatted table)
df.count()            # total row count (triggers a job!)
df.columns            # list of column names
```

Example output of printSchema() on Chicago crimes:

```
root
 |-- ID: integer (nullable = true)
 |-- Date: string (nullable = true)
 |-- Primary Type: string (nullable = true)
 |-- District: integer (nullable = true)
 |-- Arrest: boolean (nullable = true)
 |-- Domestic: boolean (nullable = true)
 |-- Year: integer (nullable = true)
```

Tip

`show(5, truncate=False)` prevents column truncation for wide data.

DataFrame = Distributed Table

ID	Date	Primary Type	District	Arrest
10001	01/01/2020	THEFT	8	false
10002	01/01/2020	BATTERY	1	true
10003	01/02/2020	NARCOTICS	12	true

⋮

Partition 0
(Worker 1)

Partition 1
(Worker 2)

Schema = Column Names + Types

Unlike RDDs, Spark **knows** the structure.

Partitioned Across Workers

Each partition lives on a different executor in the cluster.

Outline

- 1 From RDDs to DataFrames
- 2 Creating DataFrames
- 3 DataFrame Transformations**
- 4 Spark SQL
- 5 The Catalyst Optimizer
- 6 Writing Results
- 7 RDD vs DataFrame Decision Guide
- 8 Summary

Selecting and Filtering

```
from pyspark.sql.functions import col

# Select specific columns
df.select("Primary Type", "District", "Arrest").show(5)

# Filter rows
df.filter(col("Arrest") == True).count()
df.filter(col("District") == 8).show(5)

# Multiple conditions (use & for AND, | for OR)
df.filter(
    (col("Arrest") == True) & (col("District") == 8)
).count()
```

Important Syntax

In PySpark, use **parentheses** around each condition when combining with & or |. Python operator precedence requires this.

Adding and Transforming Columns

```
from pyspark.sql.functions import col, upper, length, when, lit

# Add a new column
df = df.withColumn("TypeLength", length(col("Primary Type")))

# Conditional column (like SQL CASE WHEN)
df = df.withColumn(
    "ArrestLabel",
    when(col("Arrest") == True, "Arrested")
    .otherwise("Not Arrested")
)

# Rename a column
df = df.withColumnRenamed("Primary Type", "CrimeType")

# Drop a column
df = df.drop("TypeLength")
```

Immutability

`withColumn` returns a **new** DataFrame — the original is unchanged. We reassign `df =` by convention, but no mutation occurs.

Aggregations — groupBy + agg

```
from pyspark.sql.functions import count, avg, max, min, sum

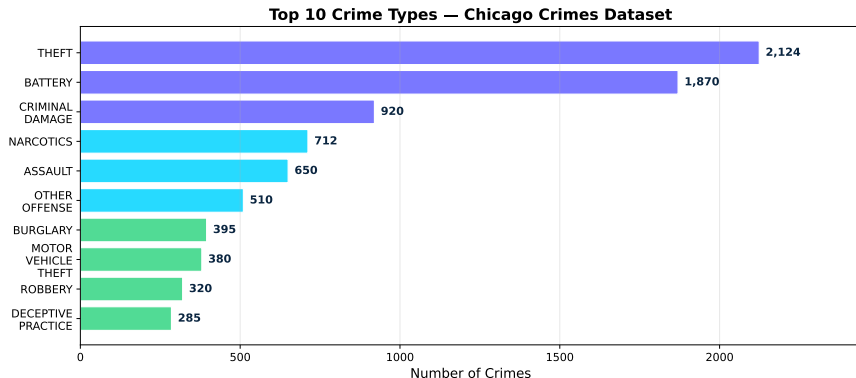
# Count crimes per type
df.groupBy("Primary Type") \
  .agg(count("*").alias("crime_count")) \
  .orderBy(col("crime_count").desc()) \
  .show(10)

# Multiple aggregations at once
df.groupBy("District") \
  .agg(
    count("*").alias("total_crimes"),
    avg(col("Arrest").cast("int")).alias("arrest_rate")
  ) \
  .orderBy("District") \
  .show()
```

Common Aggregation Functions

count, sum, avg, min, max, countDistinct, collect_list, collect_set

Aggregation Result — Visual Example



Generated from DataFrame

This chart was produced by running `df.groupby("Primary Type").count().orderBy(desc("count"))` on the Chicago Crimes dataset.

Joins — Combining DataFrames

```
# Create a small lookup DataFrame
district_df = spark.createDataFrame([
    (1, "Central"), (2, "Wentworth"),
    (8, "Chicago Lawn"), (12, "Near West")
], ["DistrictID", "DistrictName"])

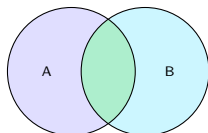
# Inner join
result = df.join(district_df,
    df["District"] == district_df["DistrictID"], "inner")
result.select("Primary Type", "DistrictName", "Arrest").show(5)

# Left join (keep all crimes)
result = df.join(district_df,
    df["District"] == district_df["DistrictID"], "left")
```

Performance Warning

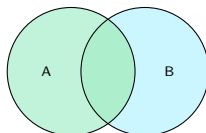
Joining two large DataFrames triggers a **shuffle**. For small lookup tables, use `broadcast(district_df)` to send it to all workers instead of shuffling.

Join Types — Visual Reference



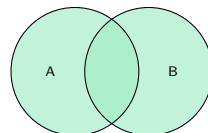
INNER JOIN

Only matching rows



LEFT JOIN

All A + matching B



FULL OUTER

All A + All B

Join Type	PySpark Keyword
Inner join	"inner" (default)
Left outer	"left" or "left_outer"
Right outer	"right" or "right_outer"
Full outer	"outer" or "full"
Cross join	"cross"

Outline

- 1 From RDDs to DataFrames
- 2 Creating DataFrames
- 3 DataFrame Transformations
- 4 Spark SQL**
- 5 The Catalyst Optimizer
- 6 Writing Results
- 7 RDD vs DataFrame Decision Guide
- 8 Summary

Registering a Temporary View

```
# Register DataFrame as a SQL table (session-scoped)
df.createOrReplaceTempView("crimes")

# Now query with standard SQL!
spark.sql("SELECT COUNT(*) FROM crimes").show()

spark.sql("""
    SELECT 'Primary Type', COUNT(*) AS crime_count
    FROM crimes
    GROUP BY 'Primary Type'
    ORDER BY crime_count DESC
    LIMIT 10
    """).show()
```

Session-Scoped

createOrReplaceTempView
Lost when SparkSession ends.

Global View

createOrReplaceGlobalTempView
Visible to other sessions.

Spark SQL vs DataFrame API — Same Result

DataFrame API:

```
df.filter(col("Arrest") == True) \
  .groupBy("Primary Type") \
  .agg(
    count("*").alias("arrests")
  ) \
  .orderBy(
    col("arrests").desc()
  )
```

Spark SQL:

```
SELECT 'Primary Type',
       COUNT(*) AS arrests
FROM crimes
WHERE Arrest = true
GROUP BY 'Primary Type'
ORDER BY arrests DESC
```

Both produce **identical** execution plans via Catalyst

Recommendation

Use whichever is clearer for your audience: **SQL** for analysts, **DataFrame API** for developers.

Spark SQL — More Examples

```
-- Top 5 districts by crime count
SELECT District, COUNT(*) AS total
FROM crimes
GROUP BY District
ORDER BY total DESC
LIMIT 5;

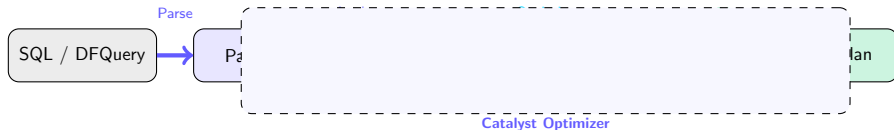
-- Arrest rate by crime type
SELECT 'Primary Type',
       ROUND(AVG(CAST(Arrest AS INT)) * 100, 1) AS arrest_pct
FROM crimes
GROUP BY 'Primary Type'
HAVING COUNT(*) > 50
ORDER BY arrest_pct DESC;

-- Crimes per year
SELECT Year, COUNT(*) AS total
FROM crimes
GROUP BY Year
ORDER BY Year;
```

Outline

- 1 From RDDs to DataFrames
- 2 Creating DataFrames
- 3 DataFrame Transformations
- 4 Spark SQL
- 5 The Catalyst Optimizer**
- 6 Writing Results
- 7 RDD vs DataFrame Decision Guide
- 8 Summary

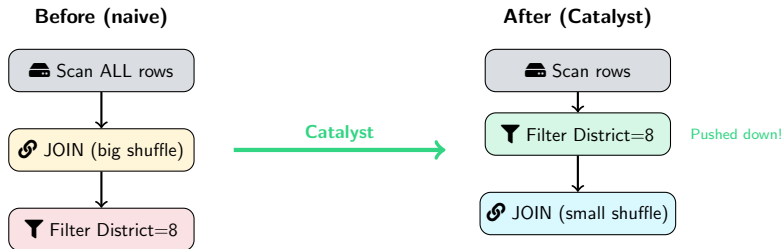
Why DataFrames Are Faster — The Catalyst Optimizer



Key optimizations Catalyst applies:

- **Predicate Pushdown:** filter *before* join (read less data)
- **Constant Folding:** evaluate constant expressions at compile time
- **Column Pruning:** read only the columns you select (skip rest)
- **Partition Pruning:** skip irrelevant Parquet/ORC partitions entirely

Predicate Pushdown — Example



Result

By filtering **before** the join, only District 8 rows are shuffled instead of all 10,000 rows. Massive I/O and network savings.

Reading the Execution Plan — explain()

```
df.groupBy("Primary Type").count().explain(True)
```

```
== Parsed Logical Plan ==
Aggregate [Primary Type], [Primary Type, count(1) AS count]
+- Relation[ID,Date,...] csv

== Optimized Logical Plan ==
Aggregate [Primary Type], [Primary Type, count(1) AS count]
+- Project [Primary Type] <-- column pruning!
   +- Relation[ID,Date,...] csv

== Physical Plan ==
HashAggregate(keys=[Primary Type], functions=[count(1)])
+- Exchange hashpartitioning <-- shuffle
   +- HashAggregate (partial) <-- local combine
      +- FileScan csv [Primary Type] <-- reads 1 col
```

Why DataFrames > RDDs

Catalyst can inspect and rewrite the plan. With arbitrary Python lambdas in RDDs, **no optimization is possible.**

Outline

- 1 From RDDs to DataFrames
- 2 Creating DataFrames
- 3 DataFrame Transformations
- 4 Spark SQL
- 5 The Catalyst Optimizer
- 6 Writing Results**
- 7 RDD vs DataFrame Decision Guide
- 8 Summary

Writing DataFrames to HDFS

```
# Write as CSV
df.write.mode("overwrite") \
  .csv("hdfs:///output/crimes_result_csv")

# Write as Parquet (recommended for downstream Spark/Hive)
df.write.mode("overwrite") \
  .parquet("hdfs:///output/crimes_result_parquet")

# Write as JSON
df.write.mode("overwrite") \
  .json("hdfs:///output/crimes_result_json")

# Partition output by a column (efficient for queries)
df.write.partitionBy("District") \
  .mode("overwrite") \
  .parquet("hdfs:///output/crimes_partitioned")
```

Mode	Behavior
overwrite	Delete existing data, write new
append	Add rows to existing data
ignore	Do nothing if data already exists
error	Throw error if data exists (default)

File Format Comparison

Format	Schema	Splittable	Compression	Best For
CSV	No	Yes	Poor	Exchange
JSON	Self	Yes	Poor	APIs
Parquet	Embedded	Yes	Excellent	Analytics
ORC	Embedded	Yes	Excellent	Hive

Recommendation

Use **Parquet** for Spark-to-Spark workflows. It's columnar, compressed, and stores the schema — no `inferSchema` needed on read.

Outline

- 1 From RDDs to DataFrames
- 2 Creating DataFrames
- 3 DataFrame Transformations
- 4 Spark SQL
- 5 The Catalyst Optimizer
- 6 Writing Results
- 7 RDD vs DataFrame Decision Guide**
- 8 Summary

RDD vs DataFrame — When to Use Which

	RDD	DataFrame
Schema	None (opaque Python objects)	Named + typed columns
Optimization	None (your code = execution)	Catalyst + Tungsten
API Style	Functional (lambdas)	Declarative (SQL-like)
Performance	Good	Better (often 2–5× faster)
Use When	Custom objects, unstructured text, graph data	Structured / semi-structured data
Debugging	<code>toDebugString</code>	<code>explain(True)</code>

Rule of Thumb

If your data has **rows and columns** (tabular), **always use DataFrames**. Use RDDs only when you need low-level control over partitioning or custom objects.

Side-by-Side: RDD vs DataFrame vs SQL

Task: Count crimes per type, sorted descending

RDD API:

```
rdd.filter(  
  lambda x: x != header  
)  
.map(  
  lambda x: x.split(",")  
)  
.filter(  
  lambda f: len(f) > 5  
)  
.map(  
  lambda f: (f[5], 1)  
)  
.reduceByKey(  
  lambda a, b: a + b  
)  
.sortBy(  
  lambda x: x[1],  
  ascending=False  
)
```

DataFrame API:

```
df.groupBy(  
  "Primary Type"  
)  
.agg(  
  count("*").alias(  
    "crime_count"  
  )  
)  
.orderBy(  
  col("crime_count")  
  .desc()  
)
```

Spark SQL:

```
SELECT  
  'Primary Type',  
  COUNT(*) AS cnt  
FROM crimes  
GROUP BY  
  'Primary Type'  
ORDER BY cnt DESC
```

	RDD	DataFrame	SQL
Lines of code	14	7	6
Optimizer	✗	✓	✓
Schema aware	✗	✓	✓

Outline

- 1 From RDDs to DataFrames
- 2 Creating DataFrames
- 3 DataFrame Transformations
- 4 Spark SQL
- 5 The Catalyst Optimizer
- 6 Writing Results
- 7 RDD vs DataFrame Decision Guide
- 8 Summary**

Key Takeaways

- 1 **DataFrames** = distributed tables with schema; built on RDDs but optimized
- 2 **SparkSession** replaces SparkContext as the entry point
- 3 Load data: `spark.read.csv/json/parquet` with schema inference or explicit schema
- 4 **Transformations**: `select`, `filter`, `groupBy`, `agg`, `join`, `withColumn`
- 5 **Spark SQL**: register a temp view, query with standard SQL
- 6 **Catalyst optimizer**: predicate pushdown, column pruning, constant folding
- 7 **Parquet** is the recommended output format for analytics pipelines



The Golden Rule

DataFrame API and Spark SQL produce **identical plans**.
Choose the style that is clearest for your team.

What's Next

Week 8A: Spark on the Cluster + Performance Tuning

- Deploy jobs with `spark-submit` to YARN
- Caching with `.cache()` and `.persist()`
- Broadcast joins for small lookup tables
- Partition tuning and repartitioning

Week 8B: Spark MLlib — Machine Learning at Scale

- ML Pipeline API: Transformer, Estimator, Pipeline
- Feature engineering with `VectorAssembler`, `StringIndexer`
- Train a classifier to predict arrests in the Chicago dataset

Milestone M3 Reminder

Due end of Week 8: RDD + DataFrame + MLlib analytics on Chicago Crimes.